فاین‌تیون ParsBERT
برای دسته‌بندی اخبار فارسی

راهنمای قدم‌به‌قدم برای مبتدی‌ها — از مفهوم تا کد، بر اساس یک پروژهٔ واقعی

داده: ۴٬۸۰۰ خلاصهٔ خبر فارسی در ۶ دسته (زیرمجموعهٔ PN-Summary)

مدل: `HooshvareLab/bert-fa-zwnj-base`

کد کامل: ریپوی `parsajafarpoor/ParsBERT-` ، شاخهٔ `claude/loving-einstein-jh2u6g`

مهر ۱۴۰۵

فهرست

1. این راهنما دربارهٔ چیست؟
2. مفاهیم پایه: یادگیری ماشین، BERT و فاین‌تیون
3. آماده کردن محیط کار
4. آشنایی با داده
5. توکنایزر: تبدیل متن به عدد
6. ساختن مدل
7. آموزش مدل
8. ارزیابی: مدل چقدر خوب است؟
9. مقایسه با مدل‌های ساده‌تر
10. بهتر کردن دقت: چه کار کرد و چه کار نکرد
11. اشکالات رایج و راه‌حل‌ها
12. واژه‌نامه
13. تمرین برای شما

> **چطور از این راهنما استفاده کنید**
>
> فصل‌ها به ترتیب جلو می‌روند. هر جا کدی آمده، می‌توانید آن را در Google Colab اجرا کنید. همهٔ عددهای این راهنما از اجرای واقعی همین کدها روی همین داده آمده‌اند، نه از حدس.

۱. این راهنما دربارهٔ چیست؟

فرض کنید هزاران خبر فارسی دارید و می‌خواهید کامپیوتر خودش تشخیص دهد هر خبر دربارهٔ چیست: **اقتصاد، بین‌الملل، ورزش، علم و فناوری، سیاست** یا **سلامت**. به این کار **دسته‌بندی متن** (Text Classification) می‌گویند.

در این راهنما یک مدل زبانی فارسی آماده به اسم **ParsBERT** را برمی‌داریم و با ۳٬۰۰۰ خبر برچسب‌دار به آن یاد می‌دهیم خبرها را دسته‌بندی کند. در پایان مدل ما روی خبرهایی که هرگز ندیده حدود **۸۹٪** درست جواب می‌دهد.

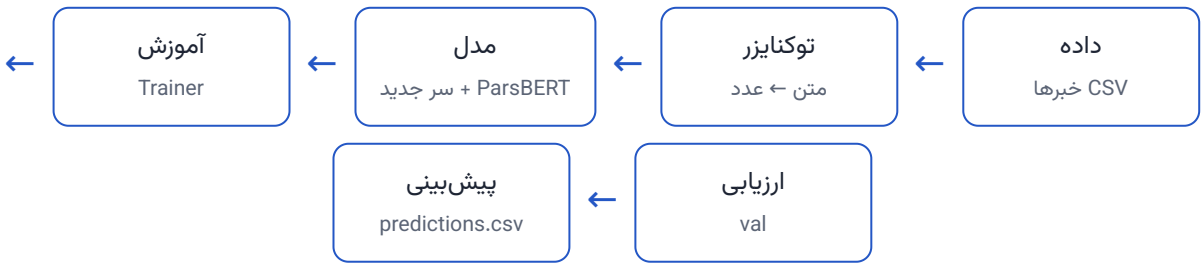
چه چیزهایی یاد می‌گیرید

- مفهوم پیش‌آموزش، فاین‌تیون و یادگیری انتقالی
- کار با کتابخانه‌های `transformers`، `pandas` و `torch`
- توکن کردن متن فارسی و دام‌های آن (مثل نیم‌فاصله)
- تنظیمات آموزش و معنای هر کدام
- خواندن نمودار یادگیری، ماتریس درهم‌ریختگی و معیارهای F1
- چطور فرضیه‌ها را با آزمایش بسنجیم، نه با حدس

پیش‌نیازها

- آشنایی مقدماتی با پایتون (متغیر، تابع، حلقه)
- یک حساب Google برای Colab (یا یک کامپیوتر با پایتون)
- نیازی به دانستن ریاضی شبکه‌های عصبی **نیست**؛ مفاهیم با مثال توضیح داده می‌شوند.

نقشهٔ کلی کار



۲. مفاهیم پایه

یادگیری ماشین با نظارت

به‌جای اینکه قانون بنویسیم («اگر کلمهٔ *فوتبال* آمد، ورزش است»)، به کامپیوتر **نمونه** نشان می‌دهیم: هزاران خبر همراه با جواب درست (برچسب). مدل خودش الگوها را پیدا می‌کند. به این کار **یادگیری با نظارت** (Supervised Learning) می‌گویند.

train، val و test: کتاب، میان‌ترم، پایان‌ترم

فایل	تعداد	کار	تشبیه
`train.csv`	3,000	مدل با این‌ها آموزش می‌بیند	کتاب درسی
`val.csv`	600	بعد از هر دور آموزش، مدل را با این‌ها امتحان می‌کنیم	امتحان میان‌ترم
`test.csv`	1,200	برچسب ندارد؛ پیش‌بینی نهایی	امتحان پایان‌ترم

> **چرا val لازم است؟**
>
> اگر مدل را فقط روی همان داده‌ای که با آن آموزش دیده بسنجیم، ممکن است فقط **جواب‌ها را حفظ کرده باشد**. val خبرهایی است که مدل ندیده؛ نمرهٔ آن نشان می‌دهد مدل واقعاً «فهمیده» یا «حفظ کرده».

شبکهٔ عصبی و BERT

شبکهٔ عصبی مجموعه‌ای از **لایه‌ها** است. هر لایه عددهای ورودی را با تعدادی **وزن** ترکیب می‌کند و به لایهٔ بعد می‌دهد. **آموزش** یعنی تنظیم این وزن‌ها تا جواب‌ها درست شوند.

BERT یک نوع خاص شبکهٔ عصبی (Transformer) است که همهٔ کلمه‌های جمله را **هم‌زمان و در ارتباط با هم** می‌بیند.
ParsBERT نسخهٔ فارسی آن است: ۱۲ لایه و حدود **۱۱۸ میلیون** وزن.

پیش‌آموزش و فاین‌تیون

ParsBERT قبلاً روی حجم زیادی متن فارسی با یک بازی ساده آموزش دیده: **«کلمهٔ جاافتاده را حدس بزن»**.

ورودی	وزیر امور `[MASK]` با همتای روسی خود `[MASK]` کرد
جواب درست	خارجه — دیدار

با این بازی، مدل زبان فارسی را یاد گرفته است. به این مرحله **پیش‌آموزش** (Pre-training) می‌گویند. ما فقط آخرین لایه را عوض می‌کنیم و کل مدل را کمی با داده‌های خودمان تنظیم می‌کنیم. به این کار **فاین‌تیون** (Fine-tuning) می‌گویند.

تشبیه

یاد دادن دسته‌بندی خبر به کسی که فارسی بلد است، خیلی راحت‌تر از یاد دادن به کسی است که باید اول الفبا یاد بگیرد. ۳,۰۰۰ خبر برای یاد دادن فارسی از صفر خیلی کم است، ولی برای یاد دادن دسته‌بندی به مدلی که فارسی بلد است کافی است. به این ایده **یادگیری انتقالی** (Transfer Learning) می‌گویند.

۳. آماده کردن محیط کار

راه پیشنهادی: Google Colab

Colab یک محیط پایتون رایگان در مرورگر است که **کارت گرافیک (GPU)** هم می‌دهد. با GPU، آموزش این پروژه حدود **۳ دقیقه** طول می‌کشد؛ با CPU حدود **۳۰ دقیقه**.

1. از منوی **Runtime → Change runtime type** گزینهٔ **T4 GPU** را انتخاب کنید.
2. در یک خانه (cell) این را اجرا کنید:

```
!git clone -b claude/loving-einstein-jh2u6g https://github.com/parsajafarpoor/ParsBERT-.git
%cd ParsBERT-
!pip install -q -r requirements.txt
!nvidia-smi        # should show "Tesla T4"
```

> **هشدار Colab**
>
> اگر Colab قطع یا ری‌استارت شود، **همهٔ فایل‌ها پاک می‌شوند**. فایل‌های مهم را بلافاصله دانلود کنید: `from google.colab import files; files.download("...")`

راه دوم: کامپیوتر خودتان

```
git clone -b claude/loving-einstein-jh2u6g https://github.com/parsajafarpoor/ParsBERT-.git
cd ParsBERT-
python -m venv env                 # separate environment
source env/Scripts/activate        # Windows Git Bash (Linux/Mac: env/bin/activate)
pip install -r requirements.txt
python train.py
```

محیط مجازی (virtual environment) پوشه‌ای جدا برای کتابخانه‌های همین پروژه است تا با بقیهٔ پروژه‌ها قاطی نشود. اگر درایو C جا ندارد، می‌توانید آن را روی درایو دیگری بسازید (مثلاً `python -m venv /d/env`).

اگر کارت گرافیک NVIDIA دارید

نسخهٔ معمولی torch روی ویندوز فقط از CPU استفاده می‌کند. نسخهٔ GPU را از سایت PyTorch بگیرید:

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
# expected: 2.x.x+cu124 True
```

اگر اینترنت ضعیف است، فایل `.whl` را با دانلودمنیجر بگیرید و با `pip install "path/to/file.whl"` نصب کنید. بعد از آن هنگام نصب بقیهٔ کتابخانه‌ها نسخهٔ torch را قفل کنید تا pip آن را با نسخهٔ CPU عوض نکند:

```
pip install -r requirements.txt "torch==2.6.0+cu124"
```

۴. آشنایی با داده

> **قانون اول**
>
> قبل از نوشتن هر کدی، داده را ببینید.

```
import pandas as pd

train = pd.read_csv("data/train.csv")
print(train.shape)                    # (3000, 2)
print(train.head())                   # first rows
print(train["label"].value_counts())  # 500 per class
print(train.isna().sum())             # missing values: 0
print(train.duplicated().sum())       # duplicate rows: 0
print(train["text"].str.split().str.len().describe())   # words per text
```

چه فهمیدیم

بررسی	نتیجه	چرا مهم است
تعداد هر کلاس	دقیقاً ۵۰۰ در train و ۱۰۰ در val	داده متعادل است؛ «دقت» معیار قابل اعتمادی است
طول متن‌ها	میانگین ۲۷ کلمه، بیشترین ۱۴۲	طول ۱۲۸ توکن کافی است (فصل ۵)
مقدار خالی و تکراری	صفر	پاک‌سازی لازم نیست
«ي» و «ك»ـ عربی	صفر	یکسان‌سازی حروف لازم نیست

در دادهٔ نامتعادل (مثلاً ۹۰٪ یک کلاس)، مدلی که همیشه همان کلاس را بگوید دقت ۹۰٪ می‌گیرد بدون اینکه چیزی بلد باشد. آنجا باید به F1 نگاه کرد (فصل ۸).

۵. توکنایزر: تبدیل متن به عدد

مدل فقط عدد می‌فهمد. **توکنایزر** متن را به تکه‌های کوچک (**توکن**) می‌شکند و هر تکه را به شماره‌اش در واژگان مدل تبدیل می‌کند.

جمله: «مدل‌های زبانی بزرگ، متن‌های فارسی را بهتر از گذشته می‌فهمند.»

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("HooshvareLab/bert-fa-zwnj-base")
ids = tokenizer(text)["input_ids"]          # text = the sentence above
print(ids)
print(tokenizer.convert_ids_to_tokens(ids))
```

خروجی واقعی:

```
[2, 21283, 8268, 2299, 590, 3092, 1981, 4127, 1937, 2789, 1925, 2520, 18311, 2193, 121, 3]
```

توکن‌ها، به ترتیب از راست به چپ:

`[CLS]` `مدلهای` `زبانی` `بزرگ` `،` `متن` `##های` `فارسی` `را` `بهتر` `از` `گذشته` `میفه` `##مند` `.` `[SEP]`

علامت	معنی
`[CLS]`	توکن ویژهٔ اول جمله. مدل خلاصهٔ کل جمله را در آن جمع می‌کند و دسته‌بند از روی همین تصمیم می‌گیرد.
`[SEP]`	توکن ویژهٔ پایان جمله.
`##`	این تکه ادامهٔ کلمهٔ قبلی است: «متن» + «##های» = «متن‌های».
`[UNK]`	کلمهٔ ناشناخته؛ مدل نتوانسته آن را از تکه‌های واژگانش بسازد.

این جملهٔ ۱۰ کلمه‌ای شد **۱۶ توکن**: کلمه‌های کمیاب چند تکه می‌شوند و [CLS] و [SEP] هم اضافه می‌شوند.

دام نیم‌فاصله

انتظار داشتیم مدلی که اسمش «zwnj» است نیم‌فاصله را نگه دارد. خروجی چیز دیگری نشان داد:

ورودی	خروجی توکنایزر
مدل‌های	مدلهای
دانش‌آموزان	`[CLS] [UNK] [SEP]`

درس

توکنایزر نیم‌فاصله را حذف کرد و «دانش‌آموزان» کاملاً ناشناخته شد. **هیچ‌وقت رفتار ابزار را حدس نزنید؛ اجرا کنید و ببینید.**

انتخاب max_length

مدل حداکثر یک طول مشخص را می‌پذیرد و بقیه بریده می‌شود. توزیع طول توکنی متن‌های train را حساب کردیم (فایل `part1_tokenizer.py`):

آمار	مقدار
کمترین / میانه / میانگین	13 / 32 / 33.6
صدک ۹۵ / صدک ۹۹ / بیشترین	55 / 69 / 167

max_length	متن‌های بریده‌شده
32	۱۴۰۷ متن (۴۶٫۹٪) — خیلی زیاد
64	۵۲ متن (۱٫۷٪)
128	**فقط ۱ متن** — انتخاب ما

Padding پویا: جمله‌های یک دسته (batch) هم‌طول نیستند؛ کوتاه‌ترها با توکن خالی `[PAD]` پر می‌شوند. ما با `DataCollatorWithPadding` هر دسته را فقط تا طول **بلندترین جملهٔ همان دسته** پر می‌کنیم، نه تا ۱۲۸. پس max_length بزرگ تقریباً هزینه‌ای ندارد.

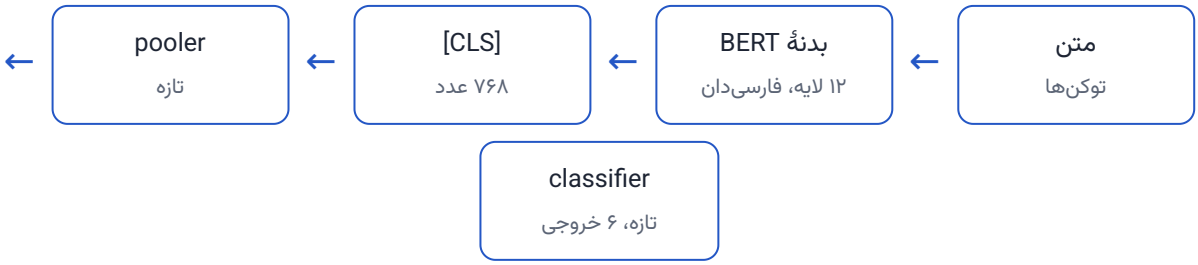
۶. ساختن مدل

```
from transformers import AutoModelForSequenceClassification

labels = sorted(train["label"].unique())
label2id = {name: i for i, name in enumerate(labels)}   # label name -> number
id2label = {i: name for name, i in label2id.items()}

model = AutoModelForSequenceClassification.from_pretrained(
    "HooshvareLab/bert-fa-zwnj-base",
    num_labels=6, id2label=id2label, label2id=label2id,
)
```

مدل از دو بخش ساخته شده



جدول UNEXPECTED / MISSING

هنگام بارگذاری یک جدول هشدار می‌بینید. **نترسید، طبیعی است:**

وضعیت	یعنی	نمونه
UNEXPECTED	در فایل مدل بود ولی لازمش نداریم؛ دور ریخته می‌شود	`cls.predictions.*`
MISSING	لازمش داریم ولی در فایل نبود؛ تصادفی ساخته می‌شود	`classifier.*, bert.pooler.*`

`cls.predictions` «سرِ حدس کلمه» از پیش‌آموزش است. ما آن را با «سرِ دسته‌بندی» عوض می‌کنیم. چون نام هیچ‌کدام از وزن‌های بدنه (`bert.encoder...`) در جدول نیست، بدنه کامل بار شده است.

چند پارامتر؟

```
total = sum(p.numel() for p in model.parameters())            # 118,301,958
head  = sum(p.numel() for p in model.classifier.parameters())  # 4,614 = 768*6 + 6
```

هر پارامتر یک عدد قابل تنظیم است. سر دسته‌بندی فقط ۴,۶۱۴ عدد دارد: ماتریس ۷۶۸×۶ (از بردار [CLS] به ۶ امتیاز) به‌اضافهٔ ۶ عدد bias.

۷. آموزش مدل

آماده کردن داده برای Trainer

```
import torch

class NewsDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings, self.labels = encodings, labels
    def __len__(self):
        return len(self.encodings["input_ids"])
    def __getitem__(self, i):
        item = {k: v[i] for k, v in self.encodings.items()}
        if self.labels is not None:
            item["labels"] = self.labels[i]
        return item

enc = tokenizer(train["text"].tolist(), truncation=True, max_length=128)
train_ds = NewsDataset(enc, train["label"].map(label2id).tolist())
```

وقتی Trainer می‌پرسد «خبر شمارهٔ i را بده»، این کلاس توکن‌ها و برچسب آن را برمی‌گرداند.

تنظیمات آموزش

```
from transformers import TrainingArguments, Trainer, DataCollatorWithPadding

args = TrainingArguments(
    output_dir="outputs",
    num_train_epochs=3,                 # epochs
    per_device_train_batch_size=16,     # batch size
    learning_rate=2e-5,
    weight_decay=0.01,
    warmup_steps=56,                    # 10% of 564 steps
    eval_strategy="epoch",              # evaluate on val after each epoch
    save_strategy="epoch",
    load_best_model_at_end=True,        # keep the best epoch
    metric_for_best_model="accuracy",
    fp16=torch.cuda.is_available(),     # faster on GPU
)
trainer = Trainer(model=model, args=args, train_dataset=train_ds, eval_dataset=val_ds,
                  data_collator=DataCollatorWithPadding(tokenizer),
                  compute_metrics=compute_metrics)
trainer.train()
```

تنظیم	معنی ساده	چرا این مقدار
epoch	یک بار دیدن کل دادهٔ آموزش	۳ کافی بود؛ ۵ بهتر نشد
batch size	مدل خبرها را ۱۶تا ۱۶تا می‌بیند و بعد از هر دسته خودش را اصلاح می‌کند	در حافظهٔ GPU جا می‌شود
learning rate	اندازهٔ هر اصلاح	کوچک، تا دانش قبلی خراب نشود
warmup	نرخ یادگیری در ابتدا از صفر آرام بالا می‌رود	سر تصادفی، بدنه را به‌هم نریزد
weight decay	جلوگیری از بزرگ شدن بیش از حد وزن‌ها	کمک به جلوگیری از حفظ کردن
fp16	محاسبه با دقت نیمه (۱۶ بیت)	سریع‌تر روی GPU

گام (step) یعنی یک بار اصلاح وزن‌ها. با ۳٬۰۰۰ خبر و دسته‌های ۱۶تایی: ⌈۳۰۰۰÷۱۶⌉ = **۱۸۸ گام در هر دوره** و ۱۸۸ × ۳ = **۵۶۴ گام در کل**.

loss: عدد خطای مدل

loss (در اینجا Cross-Entropy) نشان می‌دهد مدل چقدر اشتباه می‌کند؛ هرچه کمتر، بهتر. آموزش یعنی پایین آوردن loss.

> **آزمایش جالب: loss قبل از آموزش**
>
> پیش از هر گام آموزش، loss روی val برابر **۱٫۸۰۹** بود. چرا؟ سر تصادفی به هر ۶ کلاس احتمال تقریباً برابر (۱/۶) می‌دهد و Cross-Entropy برابر است با `−ln(1/6) = ln 6 ≈ 1.792` . دقت هم ۱۹٫۸٪ بود، نزدیک حدس تصادفی (۱۶٫۷٪). یعنی مدل در شروع دقیقاً به اندازهٔ شانس بلد است.

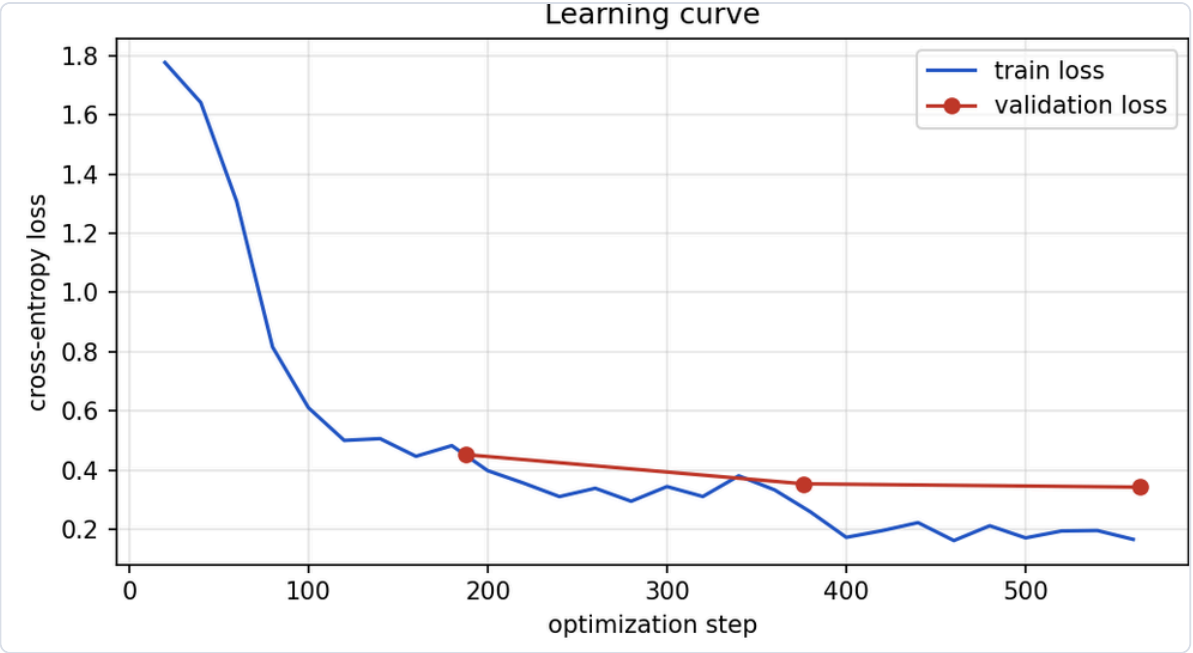
چه انتظاری داشته باشیم

```
{'loss': '1.777', 'learning_rate': '6.786e-06', 'epoch': '0.1064'}
{'loss': '0.8161', ...  'epoch': '0.4255'}
{'eval_loss': '0.4524', 'eval_accuracy': '0.84', 'eval_f1_macro': '0.8381', 'epoch': '1'}
...
{'eval_loss': '0.3426', 'eval_accuracy': '0.8883', 'eval_f1_macro': '0.8878', 'epoch': '3'}
```

loss باید کم‌کم پایین بیاید. اگر `nan` شد، fp16 را خاموش کنید (فصل ۱۱).

۸. ارزیابی: مدل چقدر خوب است؟

نمودار یادگیری



loss آموزش (آبی، هر ۲۰ گام) و loss اعتبارسنجی (قرمز، پایان هر دوره) — اجرای واقعی روی Tesla T4

دوره	loss آموزش	loss val	Macro-F1 val
1	0.899	0.452	0.838
2	0.341	0.354	0.877
3	0.195	0.343	0.888

نمودار را چطور بخوانیم

- **افت تند اول کار:** سر تازه به‌سرعت یاد می‌گیرد.
- **loss val بعد از دورهٔ ۲ تقریباً صاف می‌شود**، ولی loss آموزش هنوز پایین می‌رود. فاصلهٔ رو به افزایش دو منحنی یعنی شروع **بیش‌برازش** (overfitting): مدل دارد دادهٔ آموزش را حفظ می‌کند.
- نتیجه: ۳ دوره کافی است.

الگو	اسم	یعنی
loss آموزش پایین، loss val بالا یا رو به افزایش	**بیش‌برازش**	مدل حفظ کرده؛ دوره کمتر یا دادهٔ بیشتر
هر دو بالا و نزدیک هم	**کم‌برازش**	مدل ظرفیت یا آموزش کافی ندارد
هر دو پایین و نزدیک هم	**مناسب**	خوب است

دقت، precision، recall و F1

معیار	سؤالی که جواب می‌دهد
Accuracy (دقت)	از همهٔ خبرها، چند درصد درست دسته‌بندی شد؟
Precision	از خبرهایی که مدل گفت «سیاسی است»، چند درصد واقعاً سیاسی بودند؟
Recall	از خبرهای واقعاً سیاسی، مدل چند درصد را پیدا کرد؟
F1	میانگین متعادل precision و recall
Macro-F1	میانگین F1 همهٔ کلاس‌ها با وزن برابر

نتیجهٔ مدل نهایی روی val

کلاس	precision	recall	F1
ورزش	0.990	1.000	0.995
سلامت	0.922	0.950	0.936
اقتصاد	0.887	0.860	0.873
علم و فناوری	0.838	0.880	0.859
بین‌الملل	0.827	0.860	0.843
سیاست	0.867	0.780	0.821
کل	**Accuracy 0.888**		**Macro-F1 0.888**

ماتریس درهم‌ریختگی

سطر = برچسب واقعی، ستون = پیش‌بینی مدل. عددهای روی قطر درست‌اند و بقیه اشتباه.

	اقتصاد	بین‌الملل	سلامت	سیاست	علم	ورزش
اقتصاد	**86**	2	1	2	**9**	0
بین‌الملل	2	**86**	0	**8**	4	0
سلامت	3	0	**95**	0	2	0
سیاست	5	**14**	1	**78**	2	0
علم و فناوری	1	2	**6**	2	**88**	1
ورزش	0	0	0	0	0	**100**

بیشترین اشتباه: **سیاست ↔ بین‌الملل** (۱۴ + ۸). بعد **اقتصاد ← علم و فناوری** (۹) و **علم و فناوری ← سلامت** (۶). ورزش بی‌نقص است، چون کلمه‌هایش (لیگ، تیم، گل) خیلی مشخص‌اند.

۹. مقایسه با مدل‌های ساده‌تر

برای اینکه بدانیم فاین‌تیون ارزشش را داشت، باید آن را با یک مدل ساده‌تر مقایسه کنیم. به این مدل ساده‌تر **مدل پایه** (baseline) می‌گویند.

مدل پایه: TF-IDF + LogisticRegression

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

baseline = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True),
    LogisticRegression(max_iter=2000, C=10),
)
baseline.fit(train["text"], train["label"])
pred = baseline.predict(val["text"])
```

TF-IDF هر خبر را به برداری از وزن کلمه‌ها تبدیل می‌کند: کلمه‌ای که در این خبر زیاد و در بقیه کم آمده، وزن بیشتری می‌گیرد. ترتیب و معنای کلمه‌ها دیده نمی‌شود. این مدل در چند ثانیه آموزش می‌بیند.

آزمایش: BERT منجمد

اگر همهٔ لایه‌های BERT را ثابت نگه داریم و فقط classifier را آموزش دهیم چه می‌شود؟

```
for p in model.bert.parameters():
    p.requires_grad = False          # freeze BERT
# python train.py --freeze_bert --lr 1e-3 --epochs 5
```

نتیجه

مدل	Accuracy	Macro-F1	اشتباه از ۶۰۰
BERT منجمد (فقط classifier)	0.772	0.771	137
TF-IDF + LogisticRegression	0.847	0.846	92
ParsBERT فاین‌تیون کامل	**0.888**	**0.888**	**67**

- **فاین‌تیون ۲۷٪ خطای کمتری از TF-IDF دارد.** بیشترین بهبود در «علم و فناوری» و «اقتصاد» بود که با کلمهٔ کلیدی ساده جدا نمی‌شوند.
- **BERT منجمد حتی از TF-IDF بدتر شد.** نمایش‌های پیش‌آموزش برای «حدس کلمه» ساخته شده‌اند، نه «تشخیص موضوع». در این حالت loss آموزش همیشه از loss val بیشتر ماند: **کم‌برازش**.

درس

همیشه یک مدل پایهٔ ساده بسازید. اگر مدل پیچیدهٔ شما از آن بهتر نیست، یک جای کار ایراد دارد.

۱۰. بهتر کردن دقت: چه کار کرد و چه کار نکرد

نوسان تصادفی

همین کد را چند بار با **seed** (عدد شروع تصادفی‌ها) یا سخت‌افزار متفاوت اجرا کردیم:

اجرا	اشتباه از ۶۰۰	دقت
بهترین	62	89.7%
مدل نهایی	67	88.8%
بدترین	75	87.5%

درس بسیار مهم

فقط با عوض شدن seed، دقت بیش از ۲٪ تغییر کرد. پس اگر روشی دقت را ۱٪ بالا برد، **معلوم نیست واقعاً کمک کرده**.

برای مقایسهٔ منصفانه چند اجرا بگیرید و میانگین بگیرید. انتخاب «بهترین اجرا» هم کمی خوش‌بینانه است.

آزمایش‌ها

ایده	فایل	نتیجه
۵ دوره به‌جای ۳	`--epochs 5`	بهتر نشد
برابر کردن تعداد پیش‌بینی هر کلاس	`balance.py`	دقت را ۰٫۷٪ **کم** کرد
برچسب خبر تقریباً تکراری در train	`knn_fix.py`	بی‌اثر؛ BERT همه را از قبل درست گفته بود
ترکیب چند مدل (ensemble)	`ensemble.py`	روش معمولاً مطمئن برای خنثی کردن نوسان
پیش‌آموزش روی متن خبرها (TAPT)	`tapt.py`	یک اجرا: در محدودهٔ نوسان؛ نتیجهٔ قطعی نگرفتیم
یافتن برچسب‌های اشتباه	`find_label_errors.py`	ابزار برای پاک‌سازی داده

چرا دقت بالاتر نمی‌رود؟ نویز برچسب

بیشترین اشتباه بین سیاست و بین‌الملل بود. شمردن کلمه‌ها در train:

کلمه	در خبرهای سیاست	در خبرهای بین‌الملل
مجلس	166	5
آمریکا	71	118
تحریم	30	25

در داده، خبرهایی کاملاً دربارهٔ آمریکا و چین برچسب «سیاست» دارند و خبرهایی دربارهٔ ایران برچسب «بین‌الملل». این دیتاست از چند خبرگزاری جمع شده و برچسب بیشتر به **بخشی از خبرگزاری** بستگی دارد که خبر را منتشر کرده، نه فقط به متن. این اطلاعات در متن نیست، پس **هیچ مدلی** نمی‌تواند این اشتباه‌ها را کامل رفع کند. TF-IDF هم دقیقاً روی همین جفت بیشترین اشتباه را داشت.

> **درس**
>
> وقتی مدل‌های مختلف روی همان نمونه‌ها اشتباه می‌کنند، احتمالاً مشکل از **داده** است، نه مدل. به‌جای پیچیده‌تر کردن مدل، خطاها را بخوانید.

۱۱. اشکالات رایج و راه‌حل‌ها

همهٔ این‌ها در طول همین پروژه واقعاً پیش آمدند:

مشکل	علت	راه‌حل
`KeyboardInterrupt`	Ctrl+C در ترمینال برنامه را قطع می‌کند، کپی نمی‌کند	برای کپی: انتخاب با موس و کلیک راست
CMD ِ ویندوز وسط اجرا متوقف می‌شود	کلیک یا انتخاب متن در CMD برنامه را مکث می‌دهد	Enter یا Esc بزنید؛ روی پنجره کلیک نکنید
`can't open file 'train.py'`	ترمینال در پوشهٔ دیگری است	`cd ParsBERT-` و بعد `ls`
آدرس با `\` در Git Bash کار نمی‌کند	Git Bash از `/` استفاده می‌کند	`D:\files` ← `/d/files`
آموزش با وجود کارت گرافیک کند است	نسخهٔ torch ِ CPU نصب است	`torch.cuda.is_available()` را چک کنید؛ نسخهٔ cu124 را نصب کنید
`CUDA out of memory`	حافظهٔ GPU کم است	`--batch_size 8`
loss برابر `nan`	مشکل fp16 روی برخی کارت‌ها (مثل GTX 16xx)	`--no_fp16`
فایل پیش‌بینی مدل خوب عوض شد	اجرای بعدی روی همان فایل نوشت	هر اجرا با `--output_dir` جدا
حروف فارسی در CSV ِ Excel خراب شد	ذخیره با کدگذاری غیر UTF-8	ذخیره به‌صورت «CSV UTF-8»
فایل‌های Colab پاک شدند	runtime قطع یا ری‌استارت شد	فایل‌های مهم را فوراً دانلود کنید
Colab یا Hugging Face در دسترس نیست	فیلتر یا تحریم	اجرا روی CPU ِ خودتان (کند ولی ممکن)؛ آینهٔ `HF_ENDPOINT`

هشدارهایی که بی‌خطرند

- جدول **UNEXPECTED / MISSING** هنگام بارگذاری مدل (فصل ۶)
- `warmup_ratio is deprecated` یا هشدارهای مشابه نسخه
- `pin_memory ... no accelerator is found` روی CPU
- `You are sending unauthenticated requests to the HF Hub`

۱۲. واژه‌نامه

اصطلاح	معنی
Fine-tuning (فاین‌تیون)	تنظیم یک مدل آماده برای یک کار خاص با دادهٔ کم
Pre-training (پیش‌آموزش)	آموزش اولیهٔ مدل روی حجم زیادی متن بدون برچسب
Transfer Learning	استفاده از دانش یک کار برای کار دیگر
Token (توکن)	تکهٔ کوچکی از متن که مدل با آن کار می‌کند
Tokenizer	ابزار تبدیل متن به توکن و عدد
[UNK] / [PAD] / [SEP] / [CLS]	توکن‌های ویژه: شروع، پایان، پرکننده، ناشناخته
Parameter / Weight	عدد قابل تنظیم داخل مدل
Logits	امتیازهای خام مدل برای هر کلاس، پیش از تبدیل به احتمال
Softmax	تبدیل امتیازها به احتمال‌هایی که جمعشان ۱ است
Loss / Cross-Entropy	عدد خطای مدل؛ هدف آموزش کم کردن آن است
Epoch (دوره)	یک بار دیدن کل دادهٔ آموزش
Batch (دسته)	چند نمونه که با هم به مدل داده می‌شوند
Step (گام)	یک بار اصلاح وزن‌ها بعد از یک دسته
Learning rate	اندازهٔ هر اصلاح وزن
Warmup	بالا بردن تدریجی نرخ یادگیری در ابتدای آموزش
Overfitting (بیش‌برازش)	حفظ کردن دادهٔ آموزش به‌جای یادگرفتن الگو
Underfitting (کم‌برازش)	یاد نگرفتن کافی
Seed	عدد شروع تصادفی‌ها؛ برای تکرارپذیری
Baseline	مدل ساده برای مقایسه
Ensemble	ترکیب پیش‌بینی چند مدل
Confusion Matrix	جدول اینکه هر کلاس با کدام کلاس اشتباه گرفته شد
Freeze (منجمد کردن)	ثابت نگه داشتن وزن‌های بخشی از مدل
GPU / CUDA	کارت گرافیک / زبان ارتباط با کارت‌های NVIDIA
Label noise (نویز برچسب)	برچسب‌های اشتباه یا ناسازگار در داده

۱۳. تمرین برای شما

1. **توکنایزر:** ۵ جملهٔ فارسی دلخواه را در `tokenizer_demo.py` بگذارید. کدام کلمه‌ها چند تکه می‌شوند؟ کدام [UNK] می‌شوند؟
2. **max_length:** مدل را با `--max_length 32` آموزش دهید. دقت چقدر افت می‌کند؟ با جدول فصل ۵ مقایسه کنید.
3. **نرخ یادگیری:** `--lr 1e-3` را امتحان کنید. نمودار یادگیری چه شکلی می‌شود و چرا؟
4. **seed:** سه بار با `--seed 1`، `2` و `3` آموزش دهید. دقت‌ها چقدر فرق دارند؟
5. **ensemble:** همان سه مدل را با `python ensemble.py outputs_s1 outputs_s2 outputs_s3` ترکیب کنید. ترکیب از تک‌تک مدل‌ها بهتر است؟
6. **تحلیل خطا:** فایل `val_errors.csv` را باز کنید و ۱۰ اشتباه را بخوانید. چندتا ابهام خبر است، چندتا برچسب اشتباه، و چندتا خطای مدل؟
7. **جمله‌های مرزی:** «باشگاه استقلال سهامش را در بورس عرضه می‌کند» — مدل چه می‌گوید؟ (`part4_analysis.py`)

> **قانون طلایی این پروژه**
>
> هر ادعا را با اجرای کد و نگاه کردن به عدد بسنجید. در این پروژه چند ایدهٔ کاملاً منطقی (نگه داشتن نیم‌فاصله، balance، knn، دورهٔ بیشتر) با آزمایش رد شدند.